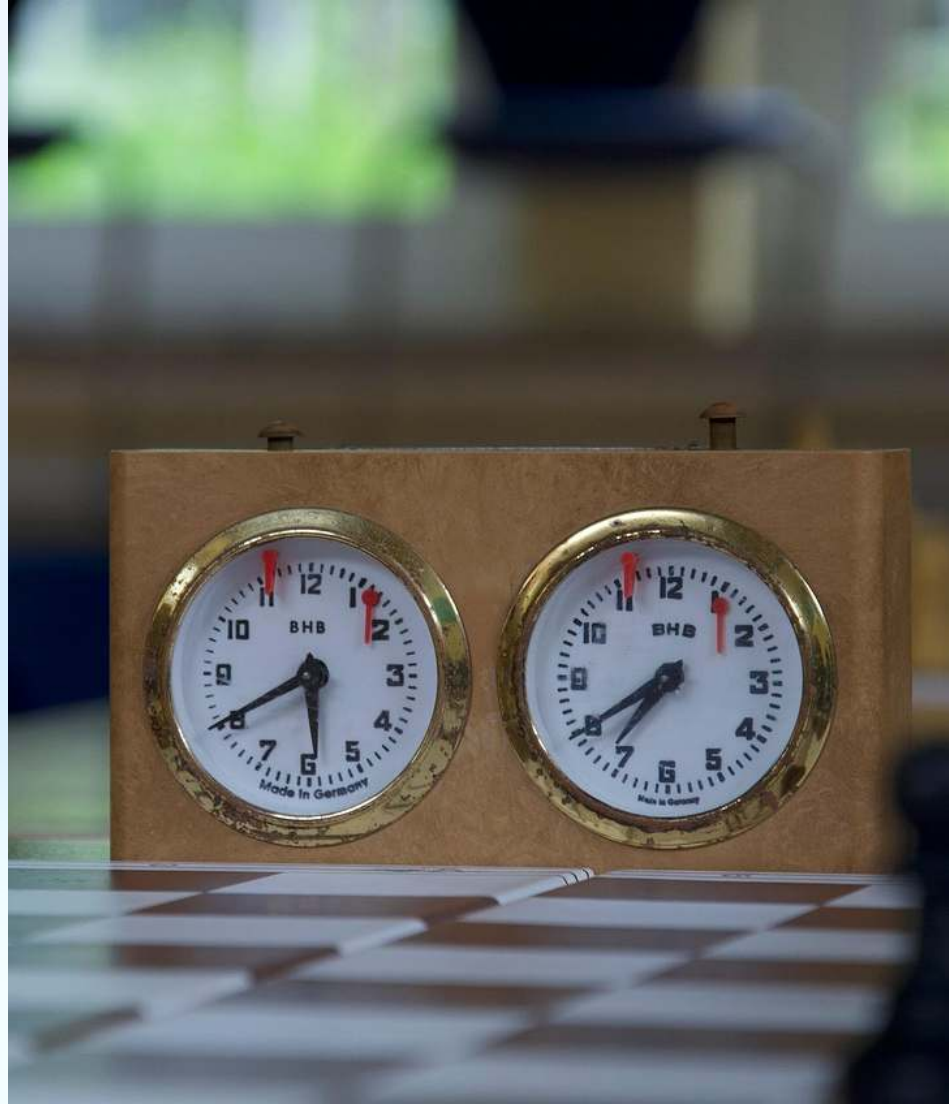


Chess Clock FSM Design

EL527 ASIC Design

Date: 2025.04.1



Presenters

Jal Dani (202201315)

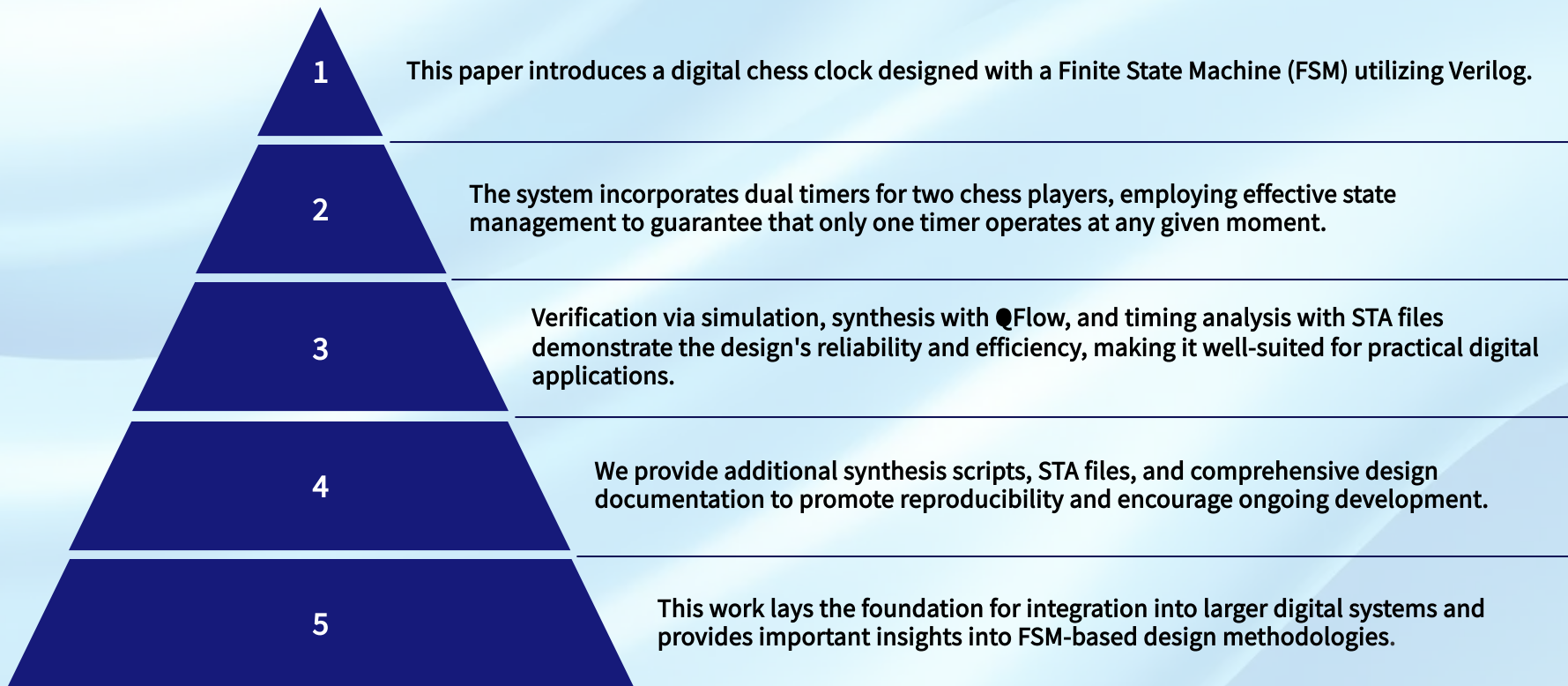
Preet Dave (202201072)

Mentor:

Dr. Rutu Parekh

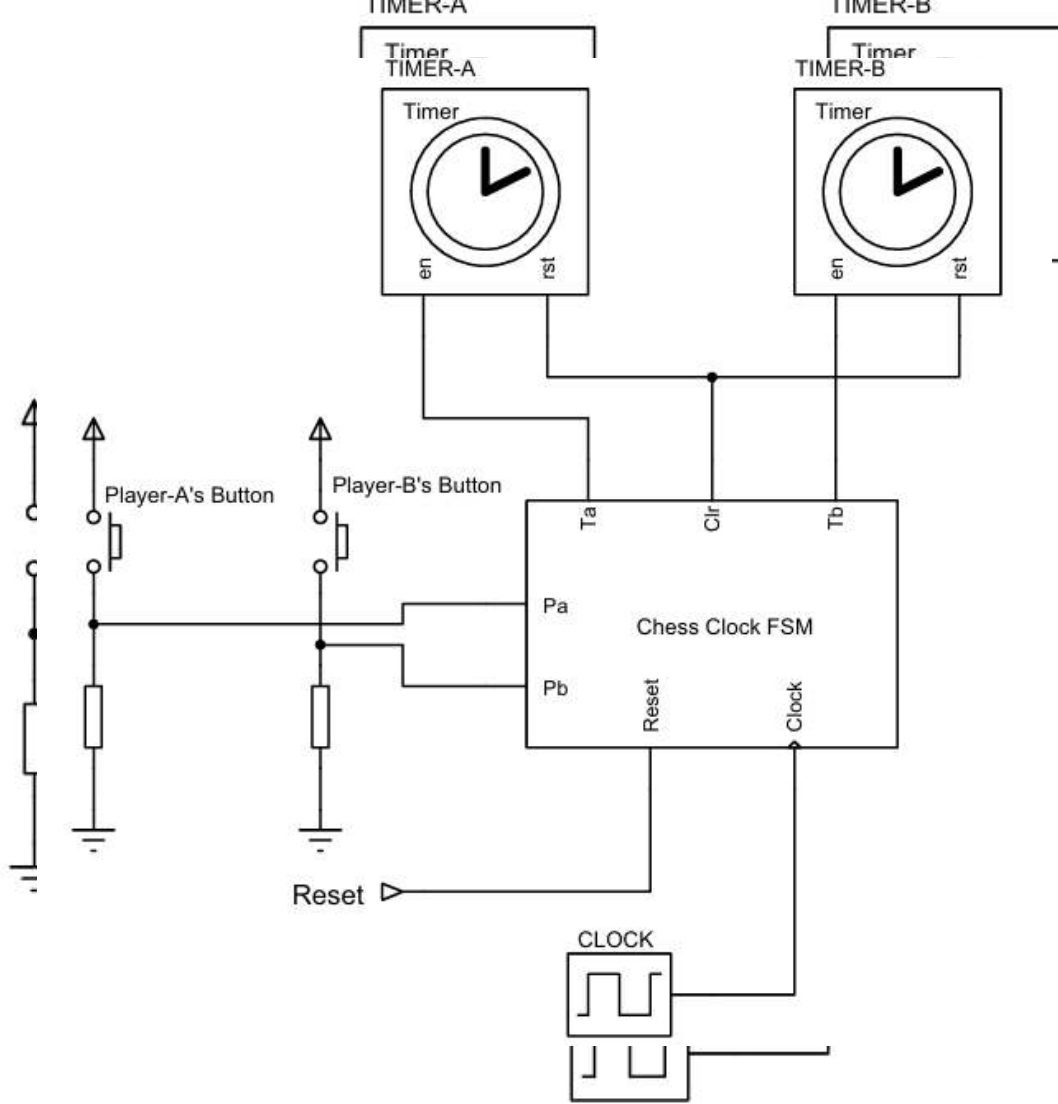


Abstract



02

Introduction



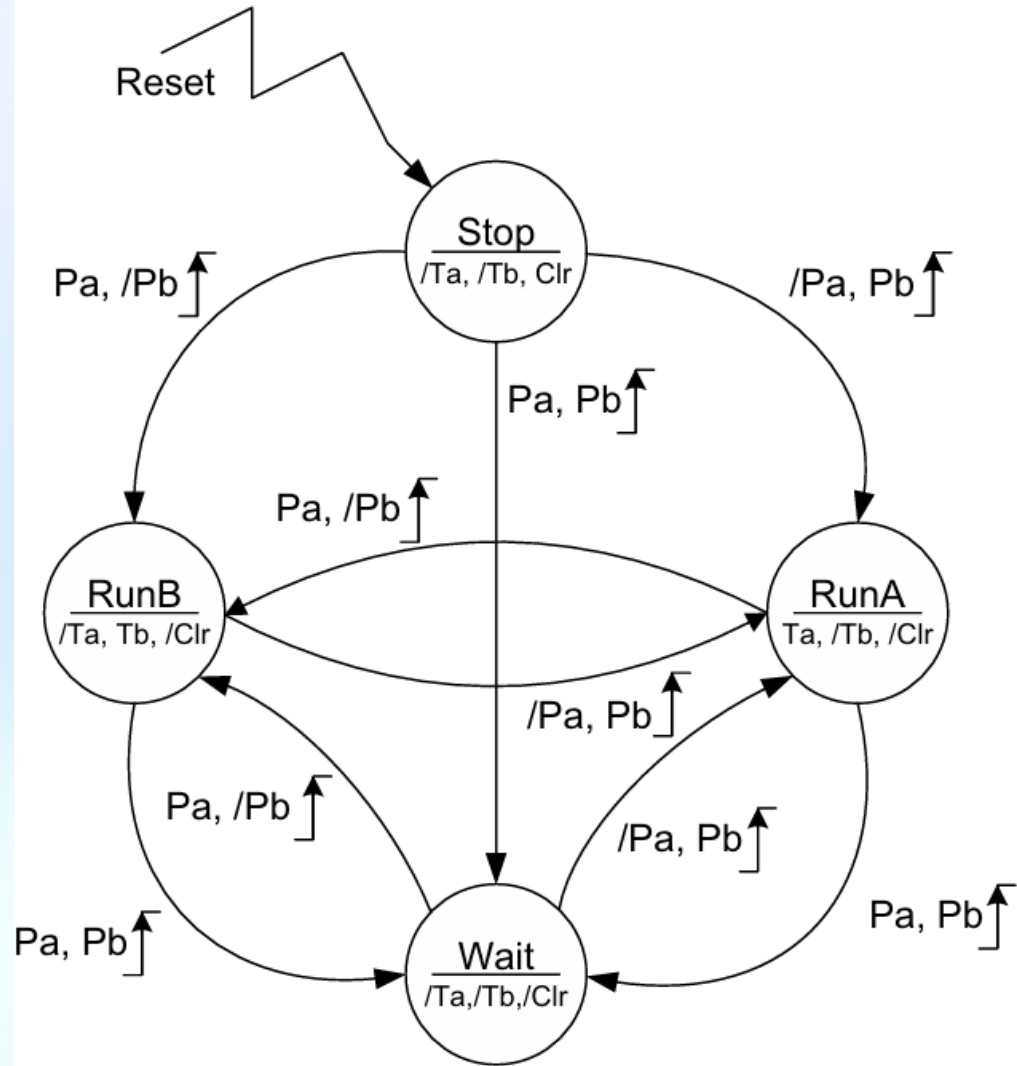
Diagram

Block Diagram

03

FSM Design

FSM Design



FSM States



Stop: Default state where both timers are inactive.



RunA: Player A's timer is active while Player B's timer remains halted.



RunB: Player B's timer is active, and Player A's timer is stopped.



Wait: Intermediate state used when both timers are paused due to a simultaneous button press.

04

RTL Implementation

RTL Implementation

Verilog Code

```
module chessclkfsm(
    input reset, Pa, Pb, clock,
);
    module chessclkfsm(
        input reset, Pa, Pb, clock,
        output Ta, Tb, Clr
    );
        // State assignments
        localparam RunA = 0, RunB = 1, Stop = 2, Wait = 3;
        reg [1:0] state;
        // State transition logic
        always @(posedge clock or posedge reset) begin
            if (reset)
                state <= Stop;
            else
                case (state)
                    RunA:
                        case ({Pa, Pb})
                            2'b0x: state <= RunA;
                            2'b10: state <= RunB;
                            2'b11: state <= Wait;
                        endcase
                    RunB:
                        case ({Pa, Pb})
                            2'bx0: state <= RunB;
                            2'b01: state <= RunA;
                            2'b11: state <= Wait;
                        endcase
                    Stop:
                        case ({Pa, Pb})
                            2'b00: state <= Stop;
                            2'b01: state <= RunA;
                            2'b10: state <= RunB;
                            2'b11: state <= Wait;
                        endcase
                    Wait:
                        if (Pa == Pb)
                            state <= Wait;
                        else if (Pa == 1'b1)
                            state <= RunB;
                        else
                            state <= RunA;
                        endcase
                endcase

            // Output logic
            assign Ta = (state == RunA);
            assign Tb = (state == RunB);
            assign Clr = (state == Stop);
        endmodule

        assign Clr = (state == Stop);
    endmodule
```



05

Explanation and Working



Understanding the Process

- 1 The RTL design efficiently manages state transitions triggered by user interactions and system resets, ensuring that only a single timer is active at any given moment to prevent conflicts and maintain accuracy.
- 2 We tested the design through simulation to make sure it works correctly and follows timing rules, so we can find and fix problems early.



06

Detailed Simulation Results

Detailed Simulation Results

- Scenario 1: Composite Waveform of All Four usrStates

Detailed Simulation Results

- Scenario 3: Stop to RunB Transition

Detailed Simulation Results

- Scenario 2: Stop to RunA Transition

Detailed Simulation Results

- Scenario 4: Extended Stop (No Player Presses)

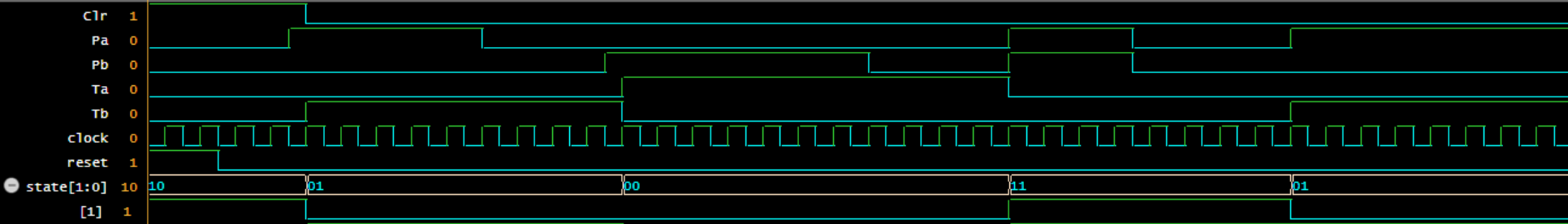
Detailed Simulation Results

- Scenario 5: Wait State (Both Buttons Pressed)

Pb

Reset

Clock



Scenario 1: All state

Figure 3 displays a comprehensive simulation trace that illustrates the FSM transitioning through all four states: Stop, RunA, RunB, and Wait.

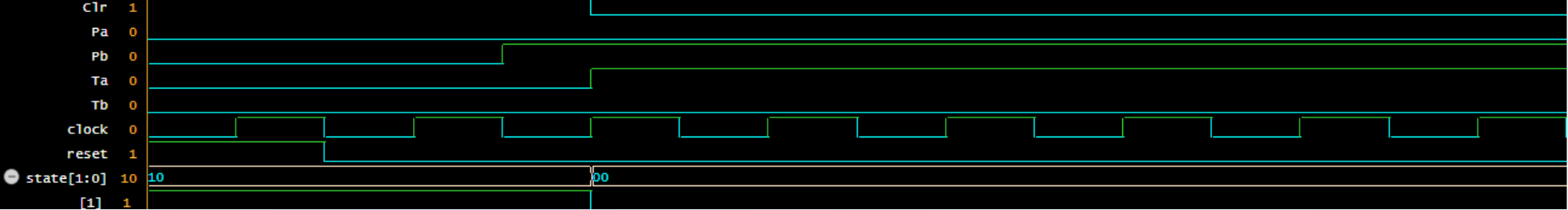
Initially, the clock is stable, and the reset signal is asserted, placing the system in the Stop state.

On the next rising clock edge when Player A (Pa) is asserted, the FSM moves to RunA, enabling Ta and disabling Tb.

When Player B (Pb) is asserted, control transitions to RunB, turning off Ta.

A simultaneous press of both buttons shifts the FSM into the Wait state, pausing both timers.

Eventually, after a timeout or reassertion of reset, the system returns to the Stop state.



Scenario 2: Stop to RunA Transition

Upon the rising edge of the clock, when Pa is asserted, the FSM transitions to RunA.

This transition activates Ta, representing Player A's timer, while Tb remains deactivated, indicating Player B's timer is idle.

The waveform confirms that only one timer is active at a time, ensuring mutual exclusivity between Ta and Tb.



Scenario 4: Extended Stop (No Player Presses)

Figure 6 demonstrates a scenario where neither Pa nor Pb is asserted after the reset is deasserted.

As a result, the FSM remains in the Stop state throughout the simulation.

Both timers remain inactive: Ta = 0, Tb = 0, and Clr = 1. Both timers remain inactive: Ta = 0, Tb = 0, and Clr = 1.

No transitions occur, indicating that the system waits for valid input before proceeding.

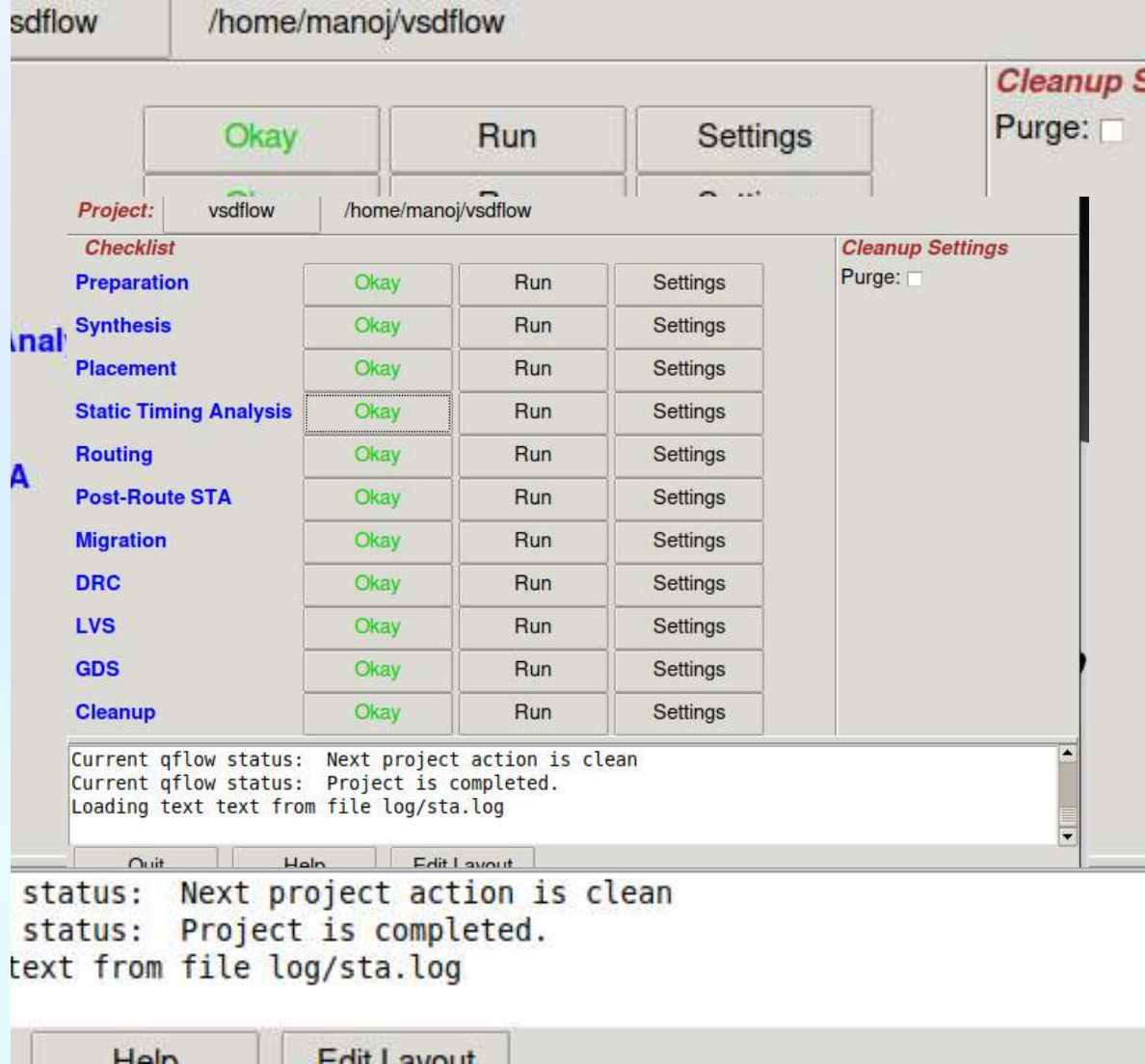
07

Implementation Using QFlow

Qflow Flow Completion

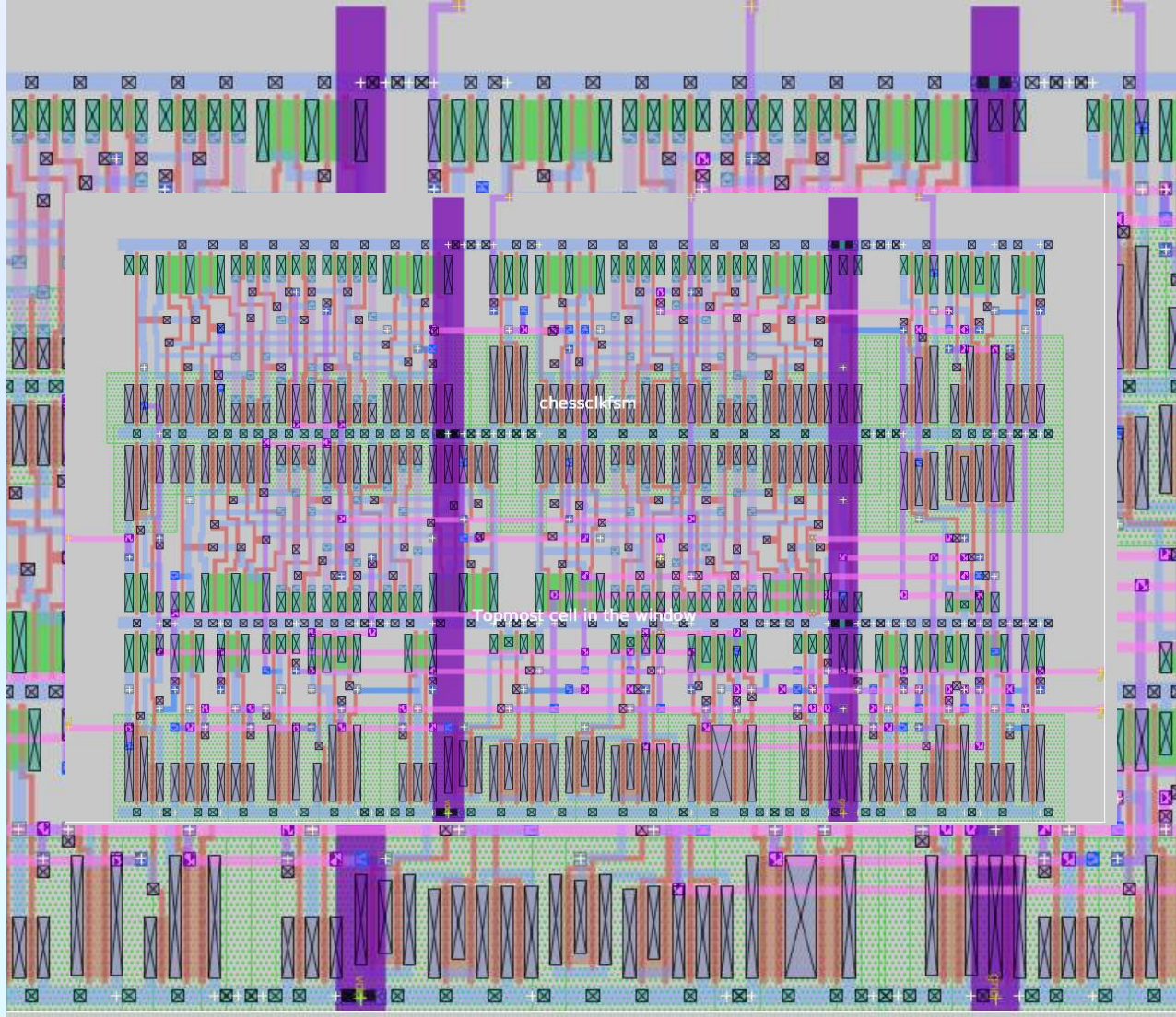
 The image showcases a successful end-to-end ASIC flow using Qflow, where every stage is marked "Okay", including

- ✓ Synthesis
- ✓ Placement & Routing
- ✓ Static Timing Analysis (STA)
- ✓ Post-Route STA
- ✓ DRC & LVS Checks
- ✓ GDSII Generation



08

Layout



09

Conclusion

Conclusion

The project successfully designed and implemented a finite state machine (FSM) for a chess clock system using Verilog and the QFlow open-source digital synthesis toolchain.

Simulation results confirmed correct functionality, and layout files were generated without errors. All codes, logs, and outputs are documented and shared for reproducibility.

Thank You